

Likha Poha AI — Platform Architecture

Version: 2.0
Last updated: 2026-07-15
Scope: Web App, Mobile App (Android), Backend API, Database, AI Layer, Auth, Payments

Table of Contents

1. System Overview
2. High-Level Architecture Diagram
3. Backend API
4. Web Frontend
5. Mobile App (Android)
6. Database Layer
7. Authentication & Authorization
8. AI / LLM Layer
9. Payments
10. Email Service
11. Deployment & Infrastructure
12. Security Architecture
13. Feature Access Matrix
14. Data Flow: Key User Journeys
15. Tech Stack Summary

1. System Overview

Likha Poha AI is a CBSE education platform targeting students in Grades 5–12. The platform delivers:

- **AI-powered lessons** — step-by-step structured lessons for every chapter
 - **Mock tests** — CBSE-style MCQ tests with instant scoring and analytics
 - **Doubt solving** — AI tutor answering any CBSE question with follow-up support
 - **Formula sheets** — subject-specific formula references
 - **Exam Prep** — NTA/JEE/NEET preparation with past papers and simulated tests
 - **Exemplar questions** — NCERT Exemplar for Grades 11/12
- The platform operates on two surfaces: a **React web app** and a **React Native Android APK**, both backed by a single **FastAPI backend** deployed on Railway.

2. High-Level Architecture Diagram





3. Backend API

Framework & Language

- **FastAPI** (Python 3.11) — async REST API
- **Uvicorn** ASGI server managed by **Gunicorn** worker manager
- Deployed on **Railway** PaaS via [nixpacks.toml](https://nixpacks.com) auto-build
- Base URL: <https://likha-poha-ai-production.up.railway.app>

Route Domains

Route Prefix	Domain	Key Endpoints
/api/auth	Authentication	/me, /signup-free, /oauth/complete-profile, /profile, /forgot-password

Route Prefix	Domain	Key Endpoints
/api/lesson	Lesson generation	/generate, /audio
/api/syllabus	Syllabus data	GET / (full grade→board→subject→chapter tree)
/api/doubt	Doubt solving	/answer, /history, /suggestions
/api/mock-test	Mock tests	/generate
/api/analytics	Analytics	/test-history (GET/POST), /leaderboard, /wrong-answers
/api/subscription	Subscriptions	/features, /plans, /upgrade, /check
/api/payments	Razorpay	/create-order, /verify, /webhook
/api/exam-prep	Exam Prep Centre	/subjects, /papers, /simulate, /submit
/api/rag	NCERT Exemplar RAG	/search, /upload, /status
/api/formula	Formula Sheets	/sheets, /get
/api/admin/*	Admin Panel	Lesson repair, cache management, analytics, AI studio
/api/teacher/*	Teacher Tools	Dashboards, classrooms, lesson plans, test papers
/api/parent	Parent Dashboard	Children, notifications, progress
/api/issues	Issue Reporting	/report, /list (admin)
/api/ai-studio	AI Studio	Prompt templates, model config

Middleware Stack (ordered)

1. **CORS middleware** — whitelists `likhapoha.in`, `localhost`, and mobile `likhapoha://`
2. **Rate limiter** — token-bucket per IP; stricter limits on `/auth/login` and `/auth/signup-free`
3. **Auth guard** (`get_current_user`) — validates Supabase JWT, attaches `user` object to request
4. **Audit middleware** — writes to `platform_audit_logs` for sensitive mutations

Service Layer (`app/services/`)

Service	Responsibility
<code>auth_service.py</code>	Supabase admin client, JWT decode, user/profile creation
<code>tutor_service.py</code>	Core LLM orchestration — lesson steps, doubt answers, mock test generation

Service	Responsibility
<code>openai_service.py</code>	OpenAI API wrapper with exponential retry + fallback to SambaNova
<code>ai_studio_service.py</code>	Prompt template management, model A/B testing, admin overrides
<code>rag_service.py</code>	pgvector similarity search for NCERT Exemplar content
<code>test_history_service.py</code>	Save/retrieve mock test scores from Supabase
<code>email_service.py</code>	Transactional email via Resend API with SMTP fallback
<code>prewarm_service.py</code>	Background lesson cache warming (LKB — Lesson Knowledge Base)
<code>rate_limit_service.py</code>	In-memory token-bucket limiter (per IP + per user)
<code>supabase_client.py</code>	Anon Supabase client (for client-level operations)

Key Design Principles

- 1. **Backend is authoritative** — all business rules, access control, and subscription checks happen here
- 2. **Frontend never bypasses the backend** — no direct Supabase table writes from the client (except auth)
- 3. **Idempotent payments** — payment webhooks use `razorpay_payment_id` as unique constraint
- 4. **Non-fatal email** — email failures never block account creation or test submission

4. Web Frontend

Framework & Hosting

- **React 18 + TypeScript + Vite** (build tool)
- **React Router v6** (client-side routing)
- Hosted on **Vercel** (CDN, auto-deploy from `main` branch on GitHub)
- Domain: <https://likhapoha.in>

Key Pages

Page	Route	Users
<code>StudentDashboardPage</code>	<code>/dashboard</code>	Students
<code>LessonsPage</code>	<code>/lessons</code>	Students
<code>DoubtPage</code>	<code>/doubt</code>	Students
<code>QuizPage</code>	<code>/quiz</code>	Students
<code>AnalyticsPage</code>	<code>/analytics</code>	Students
<code>ExamPrepCenterPage</code>	<code>/exam-prep</code>	Students (Gr 11/12)
<code>SubscriptionPlansPage</code>	<code>/plans</code>	All users

Page	Route	Users
ParentDashboardPage	/parent	Parents
TeacherDashboardPage	/teacher	Teachers
AdminControlPage	/admin	Admin
LoginPage	/login	Unauthenticated
SignupPage	/signup	Unauthenticated

Auth Flow (Web)

```
App.jsx mounts
→ supabase.auth.onAuthStateChange(SIGNED_IN | SIGNED_OUT)
→ SIGNED_IN: GET /api/auth/me
  → { needs_role_selection: true } → /role-select (Google OAuth onboarding)
  → { needs_role_selection: false } → Role-based dashboard routing
→ SIGNED_OUT: → /login
```

authFetch Pattern

All API calls use a shared `authFetch` helper:

```
// frontend/src/api/authFetch.js
const token = supabase.auth.getSession().access_token;
fetch(url, { headers: { Authorization: `Bearer ${token}` }, ...options })
```

Grade 11/12 Subject Filtering (Web)

The web app filters subjects in `LessonsPage.jsx` using `user.cbseSubjects` from the `/api/auth/me` response, showing only PCM/PCB/Commerce/Humanities subjects as appropriate.

Web-Only Features

- **Admin Panel** — lesson repair, AI studio, cache management, quality audits
- **Teacher Platform** — classrooms, student analytics, test paper generator, lesson planner
- **Sales Lead Page** — influencer offer code management
- **Lesson Quality Audit** — automated NCERT curriculum alignment checking
- **Platform Chat** — internal messaging (admin to students)

5. Mobile App (Android)

Framework

- **React Native 0.81.5 + Expo SDK 53** (managed workflow)
- **TypeScript** (strict mode)
- **expo-router** (file-based routing, same paradigm as Next.js App Router)
- Build output: signed release APK via Gradle

File Structure

```
mobile/
```

```

■■■ app/
■   ■■■ _layout.tsx           # Root auth state machine + tab navigator
■   ■■■ auth/
■   ■   ■■■ login.tsx         # Email/password + Google OAuth WebView
■   ■   ■■■ signup.tsx        # Free tier registration
■   ■   ■■■ role-select.tsx    # Google OAuth role/grade picker
■   ■■■ (tabs)/
■       ■■■ _layout.tsx       # Tab bar (Home | Lessons | Mock Test | Ask AI | Formula | ...)
■       ■■■ index.tsx         # Home dashboard (streak, stats, quick actions)
■       ■■■ lessons.tsx       # AI lesson generator + section card renderer
■       ■■■ mocktest.tsx      # MCQ test + auto-save to analytics
■       ■■■ doubt.tsx         # AI doubt solver + follow-up + history
■       ■■■ formula.tsx       # Formula sheet viewer
■       ■■■ analytics.tsx     # Test history + subject performance bars
■       ■■■ examprep.tsx      # Exam Prep Centre (JEE/NEET/Board)
■       ■■■ learn.tsx         # Web content in native WebView
■       ■■■ exemplar.tsx      # NCERT Exemplar (premium)
■       ■■■ account.tsx       # Profile, subscription, logout
■■■ components/
■   ■■■ AppHeader.tsx        # Logo + "Your Personal Tutor" header
■■■ lib/
■   ■■■ auth.ts               # Google OAuth + signIn/signUp/signOut
■   ■■■ supabase.ts           # Supabase client with expo-secure-store adapter
■   ■■■ authFetch.ts          # Authenticated API fetch wrapper
■   ■■■ theme.ts              # Light/dark mode theme provider
■■■ assets/
■   ■■■ logo.png              # Likha Poha AI logo
■   ■■■ icon-1024.png         # Adaptive icon (1024×1024, white background)
■   ■■■ likhapohaai.gif       # AI thinking loading animation
■   ■■■ google-logo.png       # Google OAuth button logo
■■■ constants.ts              # BRAND_COLOR (#6366f1), API_BASE_URL
■■■ app.json                  # Expo config (versionCode, bundleId, icon)
■■■ build_apk.sh               # v7.0 one-command APK builder script

```

APK Build Pipeline ([build_apk.sh v7.0](#))

1. `git pull` (fetch all latest commits)
2. Feature checklist: verify 17 critical features exist in source
3. Increment `versionCode` in `app.json` (e.g. 17 → 18)
4. `npm install` (sync `node_modules`)
5. `EXPO_PUBLIC_*` env vars injected from `.env`
6. `expo prebuild --platform android --clean`
7. Inject `android/app/src/main/res/xml/network_security_config.xml`
(trusts Zscaler corporate TLS certificates)
8. `cd android && ./gradlew assembleRelease`
9. Rename: `app-release.apk` → `likhapohaai-v{version}-build{N}.apk`
10. `git add app.json && git commit "build: bump versionCode to N" && git push`

Auth State Machine (Mobile)

```

_layout.tsx bootstrap
→ supabase.auth.getSession()
→ if session: GET /api/auth/me
    → { needs_role_selection: true } → router.replace("/auth/role-select")
    → { needs_role_selection: false } → router.replace("/(tabs)")
    → backend unreachable           → fallback: router.replace("/(tabs)")
→ if no session: → router.replace("/auth/login")

```

Google OAuth (Zscaler-safe WebView)

Why WebView instead of Chrome Custom Tab:

- Corporate networks (Zscaler) perform TLS inspection with their own cert
- Chrome Custom Tab uses Chrome's cert store → rejects Zscaler cert → **NET::ERR_CERT_AUTHORITY_INVALID**
- React Native WebView uses `network_security_config.xml` → trusts injected Zscaler cert → ■

```
Flow:
handleGoogleLogin()
  → supabase.auth.signInWithOAuth({ redirectTo: "likhapoha://" })
  → Opens NativeWebView modal
  → User authenticates with Google
  → Supabase redirects to:
    likhapoha://... OR https://likhapoha.in?code=... (fallback)
  → WebView.onShouldStartLoadWithRequest intercepts either URL
  → supabase.auth.exchangeCodeForSession(callbackUrl)
  → onAuthStateChange fires → routes to /(tabs)
```

Supabase configuration required:

- Authentication → URL Configuration → Redirect URLs → add `likhapoha://`

Grade Lock & Subject Filtering (Mobile)

```
// Free-tier students locked to their enrolled grade
const isGradeLocked = studentGrade !== null && !hasFullAccess;

// Grade 11/12 stream-based subject filtering
// Priority: cbse_subjects (from profile) → stream fallback → all subjects
const STREAM_SUBJECTS = {
  PCM:      ["Physics", "Chemistry", "Mathematics", "English", "Hindi"],
  PCB:      ["Physics", "Chemistry", "Biology", "English", "Hindi"],
  PCMB:     ["Physics", "Chemistry", "Mathematics", "Biology", "English", "Hindi"],
  Commerce: ["Mathematics", "Business Studies", "Accountancy", "Economics", "English", "Hindi"],
  Humanities: ["History", "Geography", "Political Science", "Sociology", "English", "Hindi"],
};

const effectiveSubjects = cbseSubjects.length > 0
  ? cbseSubjects
  : (STREAM_SUBJECTS[studentStream] ?? []);
const visibleSubjects = isGrade1112 && effectiveSubjects.length > 0
  ? allSubjects.filter(s => effectiveSubjects.includes(s))
  : allSubjects;
```

Mobile vs Web Feature Parity

Feature	Web	Mobile
AI Lessons (all steps)	■	■
Mock Tests + Analytics	■	■
Ask Doubt + Follow-up	■	■
Formula Sheets	■	■
Analytics Dashboard	■	■
Exam Prep Centre	■	■
NCERT Exemplar	■	■
Google OAuth	■	■ (WebView)

Feature	Web	Mobile
Dark Mode	■	■
Grade Lock (free tier)	■	■
Grade 11/12 stream filter	■	■
Admin Panel	■	■
Teacher Platform	■	■
Parent Dashboard	■	■ (coming)
Sales / Influencer	■	■

6. Database Layer

Platform: Supabase (PostgreSQL 15 + pgvector)

Supabase provides:

- **PostgreSQL** — relational data storage
- **Auth** — JWT-based authentication with Google OAuth support
- **Row Level Security (RLS)** — policies enforced at the DB level
- **Storage** — audio file storage (generated lesson audio MP3s)
- **pgvector** — vector embeddings for NCERT Exemplar RAG search

Key Tables

Table	Purpose
profiles	User profiles (role, grade, stream, subscription, access flags)
families	Family units linking parents to students
test_history	Mock test scores and analytics data
mock_test_wrong_answers	Per-question wrong answer tracking
subscription_payments	Razorpay payment records
subscription_plan_settings	Admin-configurable plan feature flags
offer_codes	Promotional codes for signup
offer_redemptions	Offer code usage tracking
lesson_kb	Lesson Knowledge Base — cached lesson content
lesson_kb_grade1112	Grades 11/12 lesson cache (separate DB)
doubt_history	Student doubt history (questions + AI answers)
formula_sheets	Formula sheet content (subject + grade)

Table	Purpose
exam_prep_papers	JEE/NEET/Board past papers
platform_audit_logs	Audit trail for sensitive admin actions
product_issue_reports	Student-reported bugs/issues
ai_studio_prompts	Configurable AI prompt templates
platform_chat_messages	Admin-to-student chat messages

profiles Table (Core)

```
profiles (  
  id uuid PRIMARY KEY (= supabase auth.users.id),  
  email text,  
  username text,  
  role text,           -- 'student' | 'parent' | 'teacher' | 'admin'  
  grade text,          -- 'Grade 5' .. 'Grade 12'  
  stream text,         -- 'PCM' | 'PCB' | 'PCMB' | 'Commerce' | 'Humanities'  
  board text,          -- 'CBSE'  
  cbse_subjects text[], -- e.g. ['Physics', 'Chemistry', 'Mathematics']  
  subscription_plan text, -- 'free' | 'nano' | 'premium' | 'family'  
  subscription_expires_at timestamptz,  
  account_status text,  
  access_cbse boolean,  -- Unlocks full lesson/doubt access  
  access_sof_* boolean, -- SOF Olympiad access flags  
  oauth_profile_complete boolean, -- False for new Google OAuth users pending role selection  
  family_id uuid,  
  parent_id uuid,  
  daily_token_limit int,  
  monthly_token_limit int  
)
```

Migration Strategy

All schema changes are tracked in [backend/migrations/](#) as timestamped SQL files (e.g. `20260707_grade1112_stream.sql`). Migrations are applied manually via the Supabase SQL editor.

7. Authentication & Authorization

Auth Provider: Supabase Auth

- **Email/password** signup and login
- **Google OAuth** (PKCE flow) — web uses Chrome, mobile uses WebView
- **Magic links / Password reset** — sent via Supabase email + Resend backup

JWT Flow

```
1. Client signs in (email or Google)  
2. Supabase issues JWT (access_token + refresh_token)  
3. Client stores JWT in:  
   - Web: Supabase JS client (localStorage)  
   - Mobile: expo-secure-store (encrypted native storage)
```

- 4. Every API request: Authorization: Bearer <access_token>
- 5. Backend: `get_current_user()` decodes JWT via Supabase admin client
- 6. Supabase validates signature + expiry
- 7. Returns user object with `user.id`, `user.email`

OAuth Profile Completion

New Google OAuth users don't have a role/grade yet. The `oauth_profile_complete` flag controls this:

```
New Google login
→ Supabase trigger: oauth_profile_complete = false
→ GET /api/auth/me → { needs_role_selection: true }
→ Client shows role/grade picker
→ POST /api/auth/oauth/complete-profile { role, grade, stream }
→ Backend sets oauth_profile_complete = true, creates profile row
→ GET /api/auth/me → { needs_role_selection: false } → Dashboard
```

Email signups bypass this (role is set at signup, `oauth_profile_complete = true`).

Authorization Layers

Layer	Mechanism
Route-level	<code>get_current_user</code> dependency — 401 if no valid JWT
Role-based	Route handlers check <code>user.role</code> (admin/teacher/student)
Subscription	<code>GET /api/subscription/features</code> returns feature flags per plan
Grade lock	Frontend reads <code>student.grade</code> from <code>/api/auth/me</code> , locks grade selector
Stream filter	Frontend reads <code>cbse_subjects/stream</code> to filter Grade 11/12 subjects
Database RLS	Supabase Row Level Security prevents cross-user data access

8. AI / LLM Layer

Providers

Provider	Model	Use Case
OpenAI	GPT-4o	Lesson generation, doubt solving (premium quality)
OpenAI	GPT-4o-mini	Mock test generation, quick doubts (cost-optimised)

Provider	Model	Use Case
SambaNova	Llama 3.3 70B	Fallback when OpenAI quota exceeded or unavailable
Ollama	Local models	Development/testing only (not in production)

Prompt Architecture

All prompts are managed via the **AI Studio** (</api/ai-studio>):

- Stored in [ai_studio_prompts](#) table
- Admin can edit prompts without code deployments
- Supports A/B testing between prompt variants
- Fallback to hardcoded prompts if DB unavailable

Lesson Generation Flow

```
POST /api/lesson/generate
→ tutor_service.generate_lesson(grade, subject, chapter, step_title)
→ Check lesson_kb cache (Supabase) → return cached if hit
→ Build prompt from ai_studio_prompts template
→ Call openai_service.generate() with retry (3 attempts)
→ On failure: fallback to SambaNova
→ Parse and validate response
→ Store in lesson_kb cache
→ Return structured lesson markdown
```

Doubt Solving Flow

```
POST /api/doubt/answer
→ tutor_service.answer_doubt(grade, subject, question, style_instruction)
→ Build contextual prompt (grade + subject + NCERT alignment)
→ Call LLM → stream or batch response
→ Save to doubt_history (user_id, question, answer)
→ Return { success: true, answer: markdown_text }
```

RAG (Retrieval-Augmented Generation)

For NCERT Exemplar questions, the platform uses **pgvector**:

```
User asks about a chapter
→ Generate embedding (OpenAI text-embedding-3-small)
→ pgvector similarity search on lesson_kb / exemplar chunks
→ Inject top-K chunks into LLM context
→ LLM generates answer grounded in NCERT content
```

9. Payments

Provider: Razorpay (India)

Signup payment flow:

1. User selects plan on SubscriptionPlansPage
2. POST /api/auth/signup-order { email, plan_key }
→ Backend creates Razorpay order (server-side, secure)
→ Returns { order_id, amount, key_id }
3. Client opens Razorpay checkout widget
4. User pays
5. Razorpay sends payment_id + signature to client
6. POST /api/auth/complete-signup { razorpay_payment_id, signature, ... }
→ Backend verifies HMAC signature
→ Creates auth user + profile with paid plan flags
→ Records payment in subscription_payments table

Subscription upgrade flow:

1. POST /api/payments/create-order { plan_key }
2. Client pays via Razorpay
3. POST /api/payments/verify { order_id, payment_id, signature }
→ Verifies signature
→ Updates profile: access_cbse = true, subscription_expires_at = ...
→ Returns updated user profile

Plans

Plan	Price	Access
Free	■0	DKB-only (limited doubts, no CBSE full access)
Nano	■99	8-day full access
Premium	■499/month	Full access, all features
Family	■799/month	Full access for up to 3 students
Offer Code	Varies	Time-limited full access

10. Email Service

Provider Chain

1. **Primary:** Resend API (RESEND_API_KEY + EMAIL_FROM_ADDRESS)
2. **Fallback:** SMTP (ALERT_SMTP_USER + ALERT_SMTP_PASSWORD)

Email Types

- **Welcome email** — sent on signup (free + paid)
- **Password reset** — sent via Supabase + Resend backup
- **Set-password** — sent for admin-created accounts (no password set)

Email failures are always non-fatal — account creation and test submission are never blocked.

11. Deployment & Infrastructure

Backend (Railway)

- **Auto-deploy:** push to `main` branch → Railway detects `nixpacks.toml` → builds and deploys
- **Procfile:** `web: gunicorn app.main:app -k uvicorn.workers.UvicornWorker`
- **Environment variables:** set in Railway dashboard (not in code)
- **Cold start:** ~5s on Railway free tier

Web Frontend (Vercel)

- **Auto-deploy:** push to `main` → Vercel builds Vite app → CDN deploy
- **Build command:** `npm run build` (Vite)
- **Environment variables:** `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, etc.

Mobile (Manual APK)

- Built on developer's MacBook Air (2017) via `bash build_apk.sh`
- Distributed via Google Drive → side-load on Android
- Not yet on Google Play Store (target: after build stability confirmed)
- APK naming: `likhapohaa-i-v1.0.0-build{N}.apk`

Database (Supabase)

- **Main database:** `dpivlbbyzlbwnwgajso.supabase.co` (Grades 5–10)
- **Grade 11/12 database:** separate Supabase project (larger lesson cache)
- Backups: manual JSON export via `scripts/backup_db.py`

12. Security Architecture

Principles

1. **Backend is the only trust boundary** — all access checks happen in FastAPI, never in the client
2. **JWT validation on every request** — no session cookies; all state is in the JWT
3. **No sensitive data in the frontend** — API keys, DB credentials stay on Railway
4. **Signature verification on payments** — HMAC-SHA256 on every Razorpay webhook

Key Security Controls

Control	Implementation
Auth	Supabase JWT (RS256) validated server-side
Rate limiting	Token-bucket per IP on auth + AI routes
Input validation	Pydantic models on all request bodies
SQL injection	Supabase client uses parameterized queries
XSS	React/RN auto-escape; no <code>dangerouslySetInnerHTML</code> in auth paths
Payment verification	HMAC-SHA256 signature check before account creation
Audit logging	<code>platform_audit_logs</code> table for role changes, payments, admin actions

Control	Implementation
Role conflict	<code>oauth_complete-profile</code> returns 409 if role already set
CORS	Explicit allowlist; no wildcard origin in production

Zscaler / Corporate WiFi Handling (Mobile)

Corporate networks with Zscaler SSL inspection intercept HTTPS and re-sign with Zscaler’s cert. The mobile APK handles this via `network_security_config.xml` which is injected during every build:

```
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system"/>          <!-- System CAs -->
      <certificates src="@raw/zscaler_cert_0"/>  <!-- Zscaler Root CA -->
      <certificates src="@raw/zscaler_cert_1"/>
      <certificates src="@raw/zscaler_cert_2"/>
    </trust-anchors>
  </base-config>
</network-security-config>
```

This is also why Google OAuth uses a **WebView** (which respects `network_security_config`) instead of Chrome Custom Tab (which uses Chrome’s own cert store).

13. Feature Access Matrix

Subscription Plans

Feature	Free	Nano (₹99)	Premium (₹499)	Family (₹799)
AI Lessons	DKB only	■ Full	■ Full	■ Full (3 students)
Mock Tests	■	■	■	■
Ask Doubt	Limited	■ Unlimited	■ Unlimited	■ Unlimited
Formula Sheets	■	■	■	■
NCERT Exemplar	■	■	■	■
Exam Prep	■	■	■	■
Analytics	■	■	■	■
Audio Lessons	■	■	■	■
Duration	Permanent	8 days	Monthly	Monthly

Grade Lock Rules

User Type	Lessons	Mock Test	Ask Doubt
Free student	Own grade only	Own grade only	Own grade only
Paid student	All grades 5-12	All grades 5-12	All grades 5-12
Grade 11/12 student	Stream subjects only	Stream subjects only	Stream subjects only
Parent	N/A	N/A	N/A
Teacher	All grades	All grades	All grades
Admin	All grades	All grades	All grades

14. Data Flow: Key User Journeys

Journey 1: New Student Signup (Email)

1. Student fills SignupPage (name, email, password, grade, stream)
2. POST /api/auth/signup-free
 - Create Supabase auth user (email_confirm=True)
 - Insert profile row (oauth_profile_complete=True, grade, stream, cbse_subjects)
 - Send welcome email (Resend)
3. Client: supabase.auth.signInWithPassword(email, password)
4. Supabase issues JWT
5. onAuthStateChanged fires → GET /api/auth/me
6. { needs_role_selection: false } → router to /(tabs) [Mobile] or /dashboard [Web]

Journey 2: Google OAuth (Existing User)

1. User taps "Sign in with Google"
2. [Mobile] WebView opens Supabase OAuth URL
[Web] Browser opens Supabase OAuth URL
3. User authenticates with Google
4. Supabase exchanges code → issues JWT
5. Redirects to likhapoha:// [Mobile] or https://likhapoha.in [Web]
6. Client calls exchangeCodeForSession(callbackUrl)
7. onAuthStateChanged fires → GET /api/auth/me
8. { needs_role_selection: false } (existing user) → Dashboard

Journey 3: Generate an AI Lesson (Mobile)

1. Student opens Lessons tab
2. GET /api/auth/me → grade="Grade 9", stream=null, cbse_subjects=[]
3. GET /api/syllabus → subject/chapter list
4. GET /api/subscription/features → { LESSONS: { allowed: true } }
5. Student selects: Grade 9 → Science → "Chapter 1: Matter in Our Surroundings"
6. Taps "Generate Lesson" (Step 1: Concept Introduction)
7. POST /api/lesson/generate { grade, subject, chapter, step_title }
 - Backend checks lesson_kb cache → miss
 - Calls OpenAI GPT-4o with NCERT-aligned prompt
 - Returns structured markdown lesson
 - Stores in lesson_kb for future hits
8. Mobile renders markdown as section cards:
 - Introduction | ■ Concept | ■ Examples | ■ Quick Check | ■ Summary

Journey 4: Submit Mock Test → Analytics

- 1. Student generates test: POST /api/mock-test/generate
- 2. Backend returns 5 MCQs with options dict {A,B,C,D} + answer key
- 3. Student answers all 5 questions
- 4. Taps "Submit Test"
- 5. Mobile calculates score locally
- 6. POST /api/analytics/test-history { username, grade, subject, percentage, ... }
→ Saved to test_history table
- 7. Analytics tab shows updated history on next visit

Journey 5: Payment Upgrade

- 1. Student visits Subscription Plans
- 2. Selects "Premium ₹499/month"
- 3. POST /api/payments/create-order { plan_key: "premium" }
→ Backend creates Razorpay order → returns { order_id, amount }
- 4. Client opens Razorpay checkout widget
- 5. Student pays with UPI/card
- 6. POST /api/payments/verify { order_id, payment_id, signature }
→ Backend verifies HMAC signature
→ Updates profile: access_cbse=true, subscription_expires_at=+30days
- 7. Student refreshes page → GET /api/auth/me shows new access flags
- 8. All lessons/doubts now fully unlocked

15. Tech Stack Summary

Complete Technology Inventory

Layer	Technology	Version	License
Web Framework	React	18.x	MIT
Web Build	Vite	5.x	MIT
Web Router	React Router	v6	MIT
Web Language	TypeScript	5.x	Apache 2.0
Web Hosting	Vercel	—	Proprietary SaaS
Mobile Framework	React Native	0.81.5	MIT
Mobile SDK	Expo	53.x	MIT
Mobile Router	expo-router	4.x	MIT
Mobile Build	Gradle	8.x	Apache 2.0
Mobile Language	TypeScript	5.x	Apache 2.0
Backend Framework	FastAPI	0.11x	MIT
Backend Language	Python	3.11	PSF
ASGI Server	Uvicorn + Gunicorn	—	MIT / MIT

Layer	Technology	Version	License
Backend Hosting	Railway	—	Proprietary SaaS
Database	PostgreSQL (Supabase)	15	PostgreSQL License
Auth	Supabase Auth	—	Apache 2.0 (open-core)
Vector Search	pgvector	—	MIT
AI (primary)	OpenAI GPT-4o	—	Proprietary
AI (fallback)	SambaNova Llama 3.3	—	Proprietary
Payments	Razorpay	—	Proprietary
Email	Resend	—	Proprietary SaaS
Version Control	GitHub	—	Proprietary SaaS
CI/CD (web)	Vercel GitHub integration	—	Proprietary
CI/CD (mobile)	Manual <code>build_apk.sh</code>	—	Custom
Icons (mobile)	@expo/vector-icons (Feather)	—	MIT
Markdown	react-native-markdown-display	—	MIT
Session storage	expo-secure-store	—	MIT
OAuth WebView	react-native-webview	—	MIT

Monorepo Structure

```

cbse-tutor-platform/
├── backend/ # FastAPI Python backend
│   ├── app/
│   │   ├── routes/ # API route modules
│   │   ├── services/ # Business logic services
│   │   ├── models/ # Pydantic schemas
│   │   ├── data/ # Static data (syllabus, plans)
│   │   ├── migrations/ # SQL migration files
│   │   ├── scripts/ # Admin scripts (seeding, audits, prewarm)
│   │   └── tests/ # pytest test suite
│   └── frontend/ # React web app
│       ├── src/
│       │   ├── pages/
│       │   ├── components/
│       │   ├── api/
│       │   └── context/
│       └── tests/ # Vitest tests
├── mobile/ # React Native Android app
│   ├── app/ # expo-router screens
│   ├── components/
│   ├── lib/
│   └── assets/
├── LikhapohaContext-docs/ # Architecture + context docs
│   ├── Architecture/
│   │   ├── PLATFORM_ARCHITECTURE.md (this file)
│   └── docs/
│       └── 01_PRODUCT_CONTEXT.md

```

Document maintained by the Likha Poha AI engineering team.

For questions, contact the platform admin or refer to [LikhapohaContext-docs/docs/](#).